

RAPPORT TECHNIQUE : JOUR 3

Benchmarks, Résultats Expérimentaux et Optimisation Post-Déploiement

Hamady Gackou
Ingénieur / Chercheur IA

February 7, 2026

Contents

1	Introduction : Passage à l'Échelle	3
2	Benchmarks de Performance (Infra Locale)	3
2.1	Configuration Matérielle et Modèles	3
2.2	Comparatif de Latence (Inférence Triplet)	3
3	Résultats Qualitatifs sur Banking77	3
3.1	Métriques de Clustering (Stage 1 + Stage 2)	3
3.2	Analyse des Erreurs de Granularité	4
4	Optimisation du "Prompt Engineering" Local	4
4.1	Problème de "Verbosité" (Chattiness)	4
4.2	Solution : Grammaire Contraignante (Grammar-Based Sampling)	4
5	Ablation Study : Utilité du Fine-Tuning Contrastif	4
6	Consommation Énergétique et Empreinte	5
7	Recommandations Finales et Roadmap	5
7.1	Recommandations Techniques	5
7.2	Prochaines Étapes (Jours 4+)	5

1 Introduction : Passage à l'Échelle

Suite à la stabilisation de l'infrastructure et du pipeline de données (traités précédemment), ce rapport documente la phase critique d'exécution des expériences sur le GPU L4. Nous analysons ici le comportement des modèles locaux (Ollama) en situation de charge réelle, la qualité du clustering obtenu sur le dataset **Banking77**, et les ajustements de performance nécessaires pour l'inférence.

2 Benchmarks de Performance (Infra Locale)

L'abandon de l'API OpenAI au profit d'une inférence locale introduit de nouvelles contraintes de latence et de mémoire.

2.1 Configuration Matérielle et Modèles

- **GPU** : NVIDIA L4 (24GB VRAM)
- **Backend** : Ollama (Server mode)
- **Quantization** : Q4_K_M (compromis vitesse/précision)

2.2 Comparatif de Latence (Inférence Triplet)

Le tableau ci-dessous présente le temps moyen pour résoudre une requête de triplet (a, c_1, c_2) sur 1000 échantillons.

Modèle	VRAM (GB)	Tokens/sec	Latence Moy. (ms)
Mistral 7B Instruct (v0.2)	6.2	85.4	112
Llama 3 8B Instruct	6.8	78.1	125
Gemma 7B Instruct	5.9	91.2	98
<i>Mixtral 8x7B (Q4)</i>	<i>26.4 (OOM)</i>	<i>N/A</i>	<i>N/A</i>

Observation : Le GPU L4 sature avec Mixtral 8x7B même en Q4. **Mistral 7B** offre le meilleur équilibre pour le pipeline *Perspective*.

3 Résultats Qualitatifs sur Banking77

3.1 Métriques de Clustering (Stage 1 + Stage 2)

Comparaison des performances entre l'embedding de base (avant fine-tuning) et après l'application du pipeline ClusterLLM complet avec Mistral 7B.

Méthode	ACC	NMI	ARI
K-Means (Embeddings Bruts)	0.421	0.583	0.312
ClusterLLM (Ollama Mistral 7B)	0.684	0.791	0.554
<i>Référence Papier (GPT-3.5)</i>	<i>0.710</i>	<i>0.815</i>	<i>0.590</i>

Table 1: Impact du Local LLM sur la qualité des clusters.

Analyse : La perte de performance due au passage de GPT-3.5 à Mistral 7B (quantifié) est marginale ($\approx -3\%$ sur l'ACC). Cela valide la viabilité de l'approche locale pour la confidentialité des données bancaires.

3.2 Analyse des Erreurs de Granularité

Le module de prédiction du nombre de clusters (k) a montré une tendance à la **sous-estimation** avec les modèles locaux.

```
1 Ground Truth k: 77
2 Predicted k (Mistral): 62 (Error: -19.4%)
3 Predicted k (Llama3): 84 (Error: +9.1%)
```

Listing 1: Logs d'analyse Granularité

Mistral a tendance à fusionner des intentions bancaires très proches (ex: *card_payment_fee_charged* vs *card_payment_not_recognized*), là où Llama 3 distingue mieux les nuances subtiles.

4 Optimisation du "Prompt Engineering" Local

Les modèles locaux sont plus sensibles au format du prompt que GPT-4. Nous avons dû réviser les templates JSON pour garantir un parsing strict.

4.1 Problème de "Verbosité" (Chattiness)

Mistral 7B tend à ajouter des phrases de politesse ("Here is the JSON you requested...") qui brisent le parser Python.

4.2 Solution : Grammaire Contraignante (Grammar-Based Sampling)

Nous avons injecté une contrainte système dans l'appel API Ollama pour forcer une sortie JSON valide.

```
1 # Avant : Prompt texte libre
2 # Apr s : Injection du format mode JSON
3 response = client.generate(
4     model="mistral:7b",
5     prompt=prompt,
6     format="json", # Force le mode JSON natif d'Ollama
7     options={"temperature": 0.1}
8 )
```

Listing 2: Patch appliqué au client Ollama

Cette modification a réduit le taux d'échec du parsing de **14% à 0.2%**.

5 Ablation Study : Utilité du Fine-Tuning Contrastif

Pour vérifier si le coût computationnel du fine-tuning (Stage 1, étape 2) est justifié, nous avons exécuté le clustering sans cette étape.

- **Sans Fine-tuning (Zero-shot LLM guide)** : NMI = 0.65
- **Avec Fine-tuning (Triplet Loss)** : NMI = 0.79

L'étape de fine-tuning, bien que coûteuse en temps (environ 45 minutes sur L4 pour Banking77), est indispensable pour projeter les relations sémantiques capturées par le LLM dans l'espace d'embedding dense.

6 Consommation Énergétique et Empreinte

Une mesure précise de la consommation a été réalisée via `nvidia-smi dmon`.

- **Pic de consommation** : 72W (Inférence continue)
- **Temps total de traitement (Dataset complet)** : 1h 12m
- **Coût énergétique estimé** : 0.086 kWh

Comparativement, l'utilisation de l'API OpenAI pour le même nombre de requêtes (sampling de triplets + paires) aurait coûté environ **\$12.50** (tarifs actuels GPT-3.5 input/output). La solution locale est amortie quasi-immédiatement.

7 Recommandations Finales et Roadmap

7.1 Recommandations Techniques

1. **Modèle : usez Llama 3 pour la Granularité.** Bien que Mistral soit plus rapide pour les triplets, Llama 3 excelle dans la tâche binaire "Same/Different" nécessaire au Stage 2. Une architecture hybride est recommandée.
2. **Batching : augmenter la taille.** Le GPU L4 est sous-utilisé en VRAM (6GB/24GB). Il est possible de doubler la taille des batchs lors du fine-tuning pour accélérer le processus.

7.2 Prochaines Étapes (Jours 4+)

- Implémentation de **vLLM** à la place d'Ollama pour supporter le batching de requêtes concurrentes et diviser le temps d'inférence par 3.
- Test sur le dataset **CLINC150** (plus grande variance d'intentions) pour valider la robustesse hors du domaine bancaire.

FIN DU RAPPORT D'EXÉCUTION.